

ECE 50877
Introduction to Applied Cryptography
Project Proposals

Aayuman Ghangurde
Purdue University

Sai Palaparthi
Purdue University

Rakib Ahsan
Purdue University

Zeke Ulrich
Purdue University

September 11, 2026

Proposal 1

Title: TPM for Linux Authentication

Problem Definition: Operating systems like Android, iOS, macOS, and Windows use a hardware security module for many functions, such as secure credential storage, biometric unlock, and verified/measured boot.

On Linux, it is rather the opposite. Out of the box, most systems only have at best secure boot enabled (with a shim), and perhaps some user applications store keys in the TPM (a hardware security module available on most modern x86 systems). On some configured systems, keys are stored in the TPM to unlock an encrypted disk.

While apps on Linux can store credentials on the TPM, this is typically of limited utility, as they tend not to involve user authentication. The situation on other operating systems is different. On iOS, apps can easily store credentials in the system store, which users can unlock securely using biometrics. For example, on Windows, password managers can securely stash a long and complex master password (used to first unlock the password database) into a system store, then allowing users to unlock the password database using biometric credentials later.

Projects such as `credentialsd` deal with userspace integration (a generic interface for providers and consumers of fido2 passkeys). However, they do not deal with secret material residing on a hardware module.

Thus we aim to answer how to provide hardware-backed credentials and trusted user verification as reusable system services on Linux, while allowing existing applications and authentication infrastructure to consume those services through standardized interfaces.

Proposed Methodology: We propose to build upon the `credentialsd` architecture by implementing a new credential provider backed by the Trusted Platform Module. Rather than storing passkeys as userspace secrets, the provider would direct the TPM to generate keys and sign challenges. The use of these credentials can be provided through a local pin, and possibly biometrics.

The TPM standard is vast, and thus, we aim to reverse-engineer, or at least determine to some extent, how Windows uses the TPM for its Windows Hello solution. Given these findings, and research into the TPM standard, we will interface with the TPM suitable for our purposes.

Further more, credentials require user verification. We will investigate reasonable ways for user credential entry, and investigate if there are secure methods available on linux as a secondary goal. Since a trusted operating system is a prerequisite for credential entry, we propose to build the TPM solution on top of verified and measured boot, for which there is significant prior work.

We will evaluate this new implementation under a comparison to Windows Hello, and interoperability with existing FIDO consumers, unauthorized usage of the private keys, integration with the standard linux logon process, and recovery behavior.

Expected Findings: We hope to document how other mainstream operating systems manage user authentication and secrets. We expect that by securing keys in a hardware security module, instead of storing them as userspace secrets, we can improve the security of credentials on Linux.

We expect to demonstrate that TPM can provide the primitives to store non-exportable FIDO2 passkeys that be used by a credential provider through credentialsd. We expect that the same mechanism can be used for Linux logon.

We expect to identify the limits of hardware-backed credential protection, and aim to characterize the limits of trusted user verification on Linux. We expect to identify the necessary components for a Windows Hello style authentication system, and the areas in which Linux infrastructure is lacking.

Proposal 2

This proposal originates with Dr. Zahra Ghodsi of Purdue University.

Title: Differential Timing Analysis for Cryptographic Library Fingerprinting

Problem Definition: The starting step of exploitation is determining the target’s technology stack. An important part of this process is identifying the cryptographic libraries and implementations running on the target. Existing tools can often identify software through explicit version information or protocol behavior, but such information may be unavailable or intentionally hidden. This motivates the following research question: Can differential timing analysis identify the cryptographic library and version used by a remote target, despite network and system noise?

Proposed Methodology:

- We will construct a testbed containing multiple implementations and versions of commonly used cryptographic libraries.
- We will first collect timing measurements while each implementation performs a controlled suite of cryptographic operations under different inputs and configurations.
- We will analyze the resulting timing distributions to identify features that distinguish implementations.
- We will then train and evaluate statistical and machine-learning classifiers using these timing features.

Expected Findings: We expect the number of samples required to train accurate classifiers to rise substantially with noise. We also expect machine learning classifiers to outperform statistical classifiers, indicating broader applicability of machine learning techniques to side-channel analysis.